# UML Design Assignment

Class, Component, Deployment and Use Case Diagrams

**Project:** BlockCred – Blockchain-Based Academic Credential Verification System

| | |
|---|---|
| **Name** | Srashti Dwivedi |
| **Roll Number** | 2501560012 |
| **Course** | MCA |
| **Specialization** | CORE |
| **Semester** | 3rd |
| **Submitted To** | Firoz Ahmed |
| **GitHub Link** | https://github.com/bcalm-2/BlockCred |

# 1. Problem Statement

Academic certificates and marksheets are still verified largely by hand: an employer or university writes to the issuing institution, waits days for a reply, and has no reliable way to tell a genuine document from a forged one. Fake degrees are a persistent problem in hiring and admissions, and students depend on paper copies or on the institution being reachable years after graduation.

**BlockCred** addresses this with a tamper-proof verification platform. When an institution issues a credential, the document is stored on IPFS and its SHA-256 hash is anchored on a blockchain through a smart contract. The student receives a QR code or secure link. A verifier scans the code or uploads the document, and the system recomputes the hash and checks it against the on-chain record, confirming authenticity and revocation status within seconds.

## Key Features

- Institution onboarding with approval by the System Administrator
- Bulk upload of student records (CSV) and credential issuance with SHA-256 hashing
- Immutable anchoring of credential hashes on an Ethereum testnet smart contract
- Off-chain document storage on IPFS, with metadata in PostgreSQL
- Student-controlled sharing through QR codes and expiring secure links
- Instant verification by employers and universities, plus revocation support
- Role-based access control, multi-factor authentication, audit logging and email notifications

## Actors

- **Student** – views credentials, shares them and controls access
- **Institution Admin** – uploads records, issues and revokes credentials
- **Verifier** (employer or university) – verifies credentials and views history
- **System Administrator** – approves institutions, manages users, reviews audit logs

## Technology Stack

React (frontend), Node.js / Express (backend), PostgreSQL (database), Solidity smart contract on Ethereum Sepolia (ethers.js), IPFS via a pinning service, Docker and Nginx for deployment.

# 2. Class Diagram

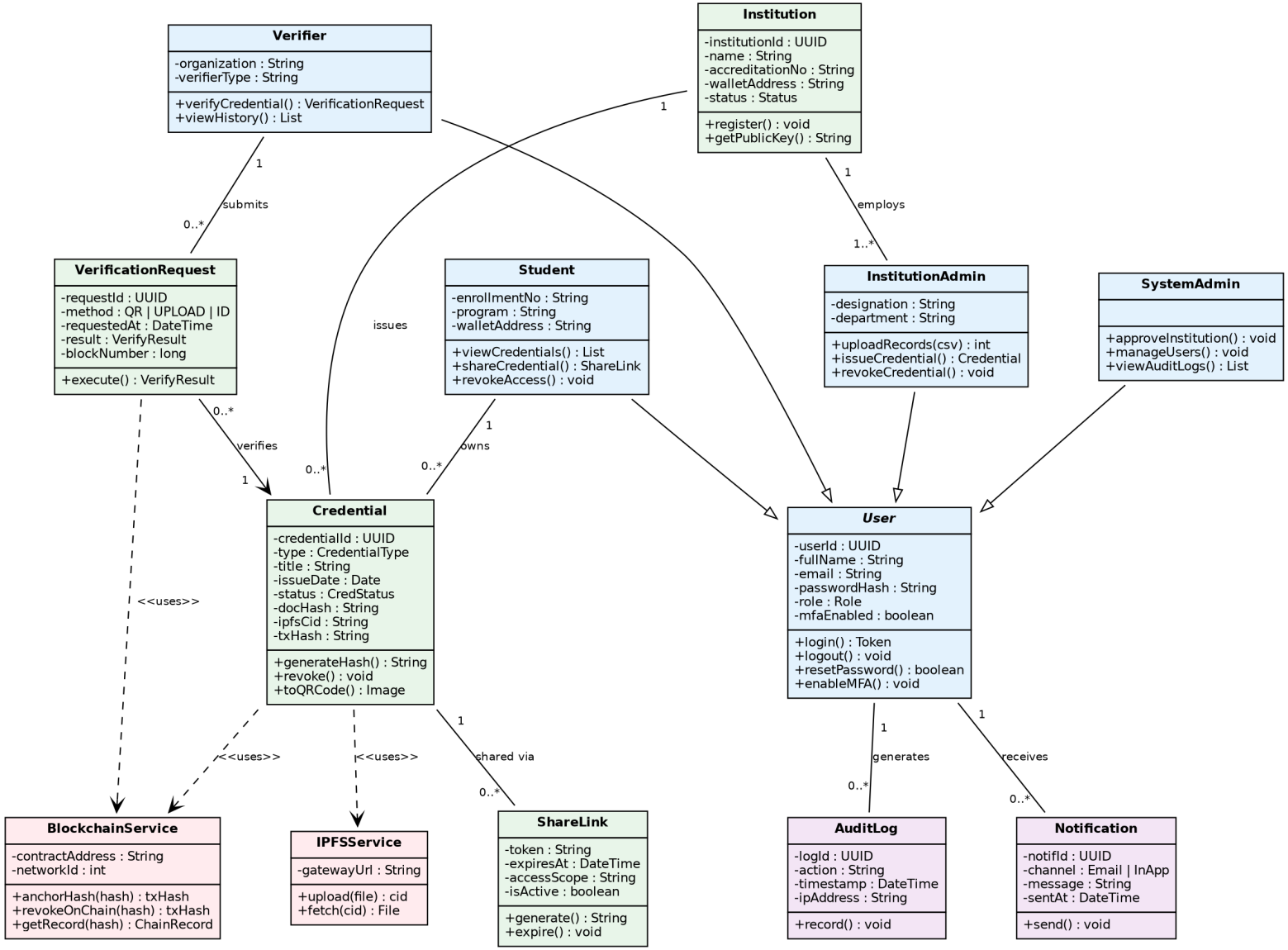



Figure 1: Class diagram of BlockCred – users (blue), core domain (green), supporting logs (purple) and service classes (red).

# 3. Component Diagram

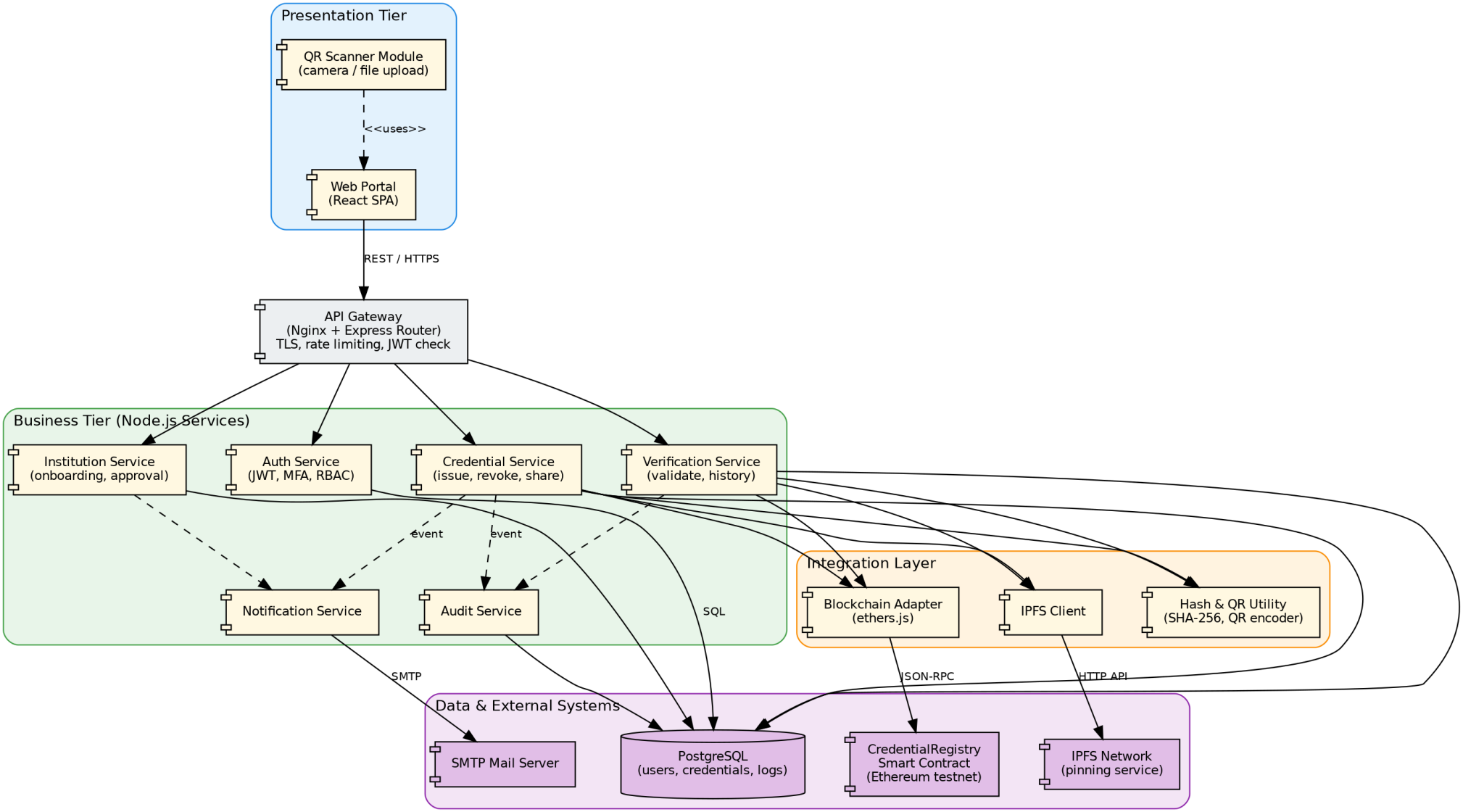



Figure 2: Component diagram showing presentation, business and integration tiers with data stores and external systems.

# 4. Deployment Diagram

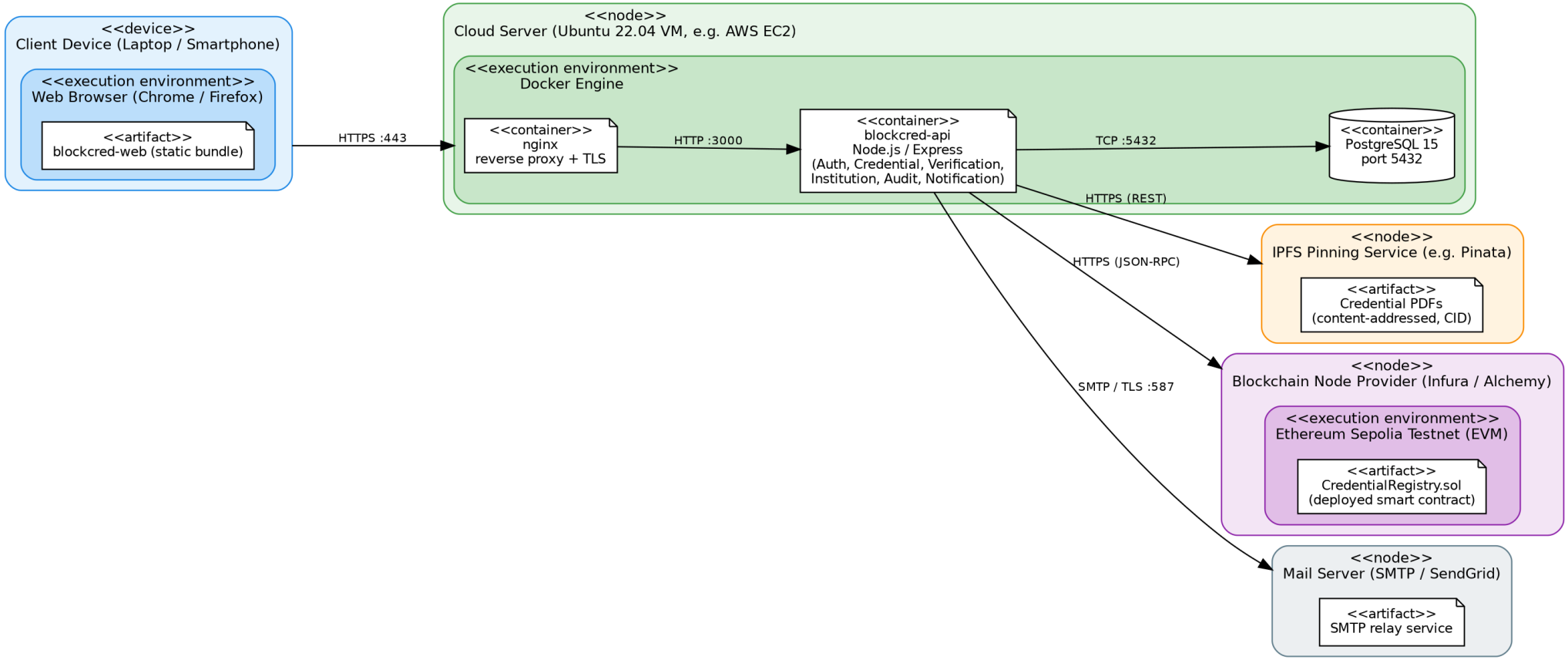



Figure 3: Deployment diagram showing nodes, execution environments, artifacts and communication protocols.

# 5. Use Case Diagram

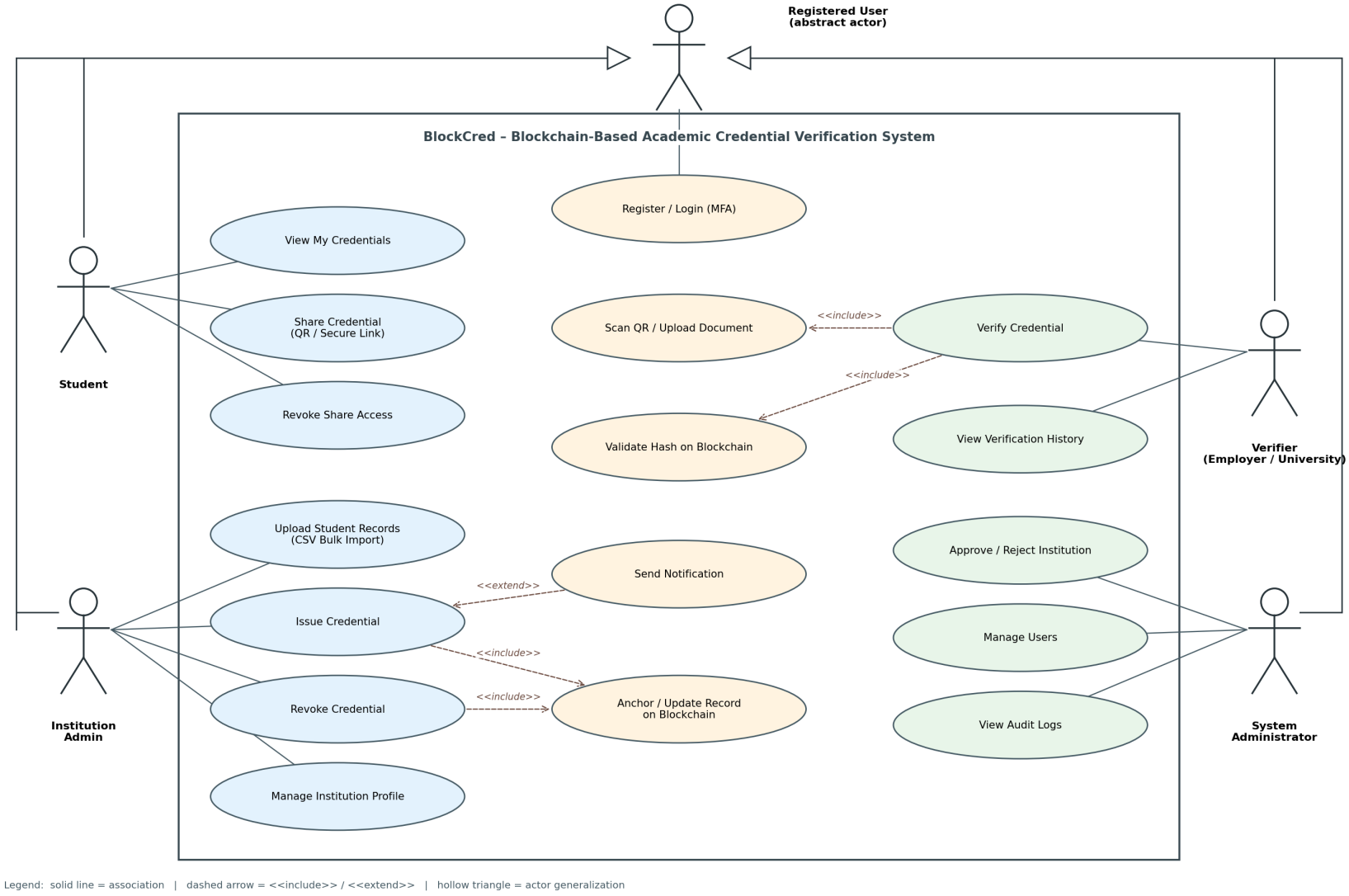



Figure 4: Use case diagram with four actors, «include» and «extend» relationships and actor generalization.

# 6. Diagram Notes

## Class Diagram

The abstract User class is specialised into Student, InstitutionAdmin, Verifier and SystemAdmin. An Institution employs one or more admins and issues many Credentials; each Credential is owned by one Student and can be exposed through many ShareLinks. A Verifier submits VerificationRequests, each pointing to one Credential. AuditLog and Notification are generated per user. BlockchainService and IPFSService are dependency classes used by Credential and VerificationRequest for anchoring, storing and validating documents.

## Component Diagram

The React web portal, with its QR scanner module, talks to an API Gateway over HTTPS. The gateway routes requests to the Auth, Institution, Credential and Verification services. The Credential and Verification services rely on the Hash & QR utility, IPFS client and Blockchain adapter, and publish events to the Notification and Audit services. Persistent data lives in PostgreSQL, documents on IPFS, hashes in the smart contract, and emails go out through an SMTP server.

## Deployment Diagram

The user's browser downloads the static web bundle and connects to a cloud VM over HTTPS on port 443. Inside Docker, Nginx terminates TLS and forwards to the Node.js API container, which uses a PostgreSQL container on the same host. The API reaches external nodes: an IPFS pinning service (HTTPS), an Ethereum Sepolia node provider hosting the CredentialRegistry contract (JSON-RPC) and an SMTP relay for notifications.

## Use Case Diagram

All four actors inherit from a Registered User actor, which is why they share the Register / Login (MFA) use case. Verify Credential includes scanning or uploading the document and validating its hash on the blockchain. Issue Credential and Revoke Credential both include anchoring or updating the record on the blockchain, and Send Notification extends Issue Credential as an optional step.